Parallelizing LLaVA

ML710 course project · MBZUAI

Rassul Magauin · Omar Ahmed · Youssef Ghallab

Spring 2026

Setup

- MBZUAI HPC, ws-ia partition: 2 nodes × 1 RTX 5000 Ada (32 GB), Ethernet ~1 Gbps.
  - No NVLink, no InfiniBand. Cross-node comm goes over a slow link.
  - The gpu partition has a 1-GPU-per-user QOS, so multi-GPU = two salloc sessions + torchrun.
- Software: PyTorch 2.1.2, DeepSpeed 0.12.6, Transformers 4.37.2, peft 0.6.0.
- Team of three, each owning at least one non-trivial parallelism strategy:
  - Pipeline parallelism — Youssef Ghallab
  - DeepSpeed ZeRO-2 — Omar Ahmed
  - DeepSpeed ZeRO-3 and PyTorch FSDP — Rassul Magauin

The model: LLaVA

- Liu et al., "Visual Instruction Tuning" (NeurIPS 2023). Multimodal: image + text → text.
- About 7B parameters total, in three pieces:
  - CLIP ViT-L/14-336 vision encoder — frozen.
  - 2-layer MLP projection — small, trainable.
  - Vicuna-7B language model — frozen in Stage 1, fine-tuned in Stage 2.
- Two-stage training:
  - Stage 1 trains only the projection (~80 MB) on 558K image-caption pairs.
  - Stage 2 fine-tunes the projection plus the LLM on 665K instructions.
- We use Stage 2 because it has meaningful gradient communication. Stage 1's AllReduce is too small to matter.

Stage 2 — what we trained

- Starting checkpoint: liuhaotian/llava-v1.5-7b. The CLIP and MLP were already trained in Stage 1.
- Dataset: 10K random samples from llava_instruct_150k.json with COCO 2014 train images. One epoch keeps each run under an hour.
- Training target: LoRA adapters (r=128, α=256) on the LLM, plus the full MLP projection.
  - ≈4.6% of total parameters — about 340M of 7.4B.
- AdamW, bf16, gradient checkpointing, cosine schedule.
- Effective batch sizes vary so each method actually fits in memory:
  - DDP and ZeRO-2: 32 (per_device=16, world=2)
  - Pipeline: 16 (per_device=1, accum=16, world=2)
  - ZeRO-3 and FSDP: 8 (per_device=1, accum=4, world=2)

What we compared

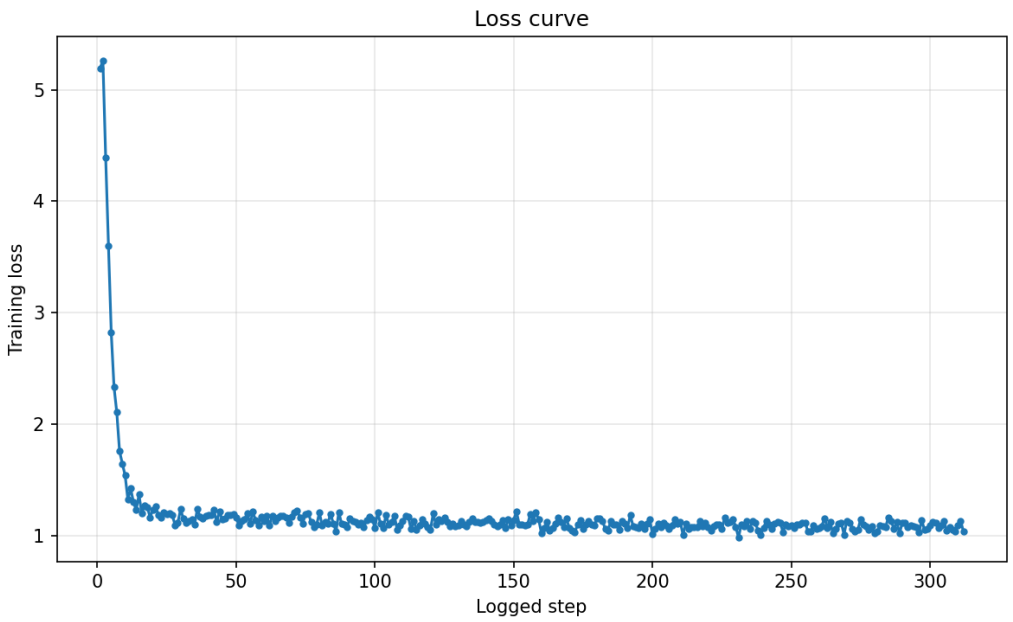
- Single-GPU LoRA — reference for what no parallelism looks like.
- DDP (DeepSpeed ZeRO-0) — naïve baseline; the model is replicated, gradients are AllReduce'd.
- Pipeline parallelism — the model is split across the two nodes. Owned by Ghallab.
- ZeRO-2 — data parallelism with sharded optimizer states and gradients. Owned by Omar.
- ZeRO-3 — same as ZeRO-2 but parameters are also sharded. Owned by Rassul.
- PyTorch FSDP — PyTorch-native equivalent of ZeRO-3, different implementation. Owned by Rassul.
- Every multi-node run uses the same two ws-ia nodes, same Ethernet link.

Baseline 1: single GPU

- One RTX 5000 Ada, batch size 1, 10000 steps for one epoch over the 10K subset.
- Wall time: 96 minutes.
- Throughput: 1.74 samples/sec.
- Peak GPU memory: 25.7 GiB out of 32. Fits comfortably.
- Average GPU utilization: 11%.
  - CPU-bound on data preprocessing — dataloader_num_workers=0, single-threaded image loading.
  - The GPU spends most of its time waiting for the next batch.
- Final loss: 1.32 (avg 1.10 across the run).
- Reference for what 'no parallelism' looks like before we add anything.

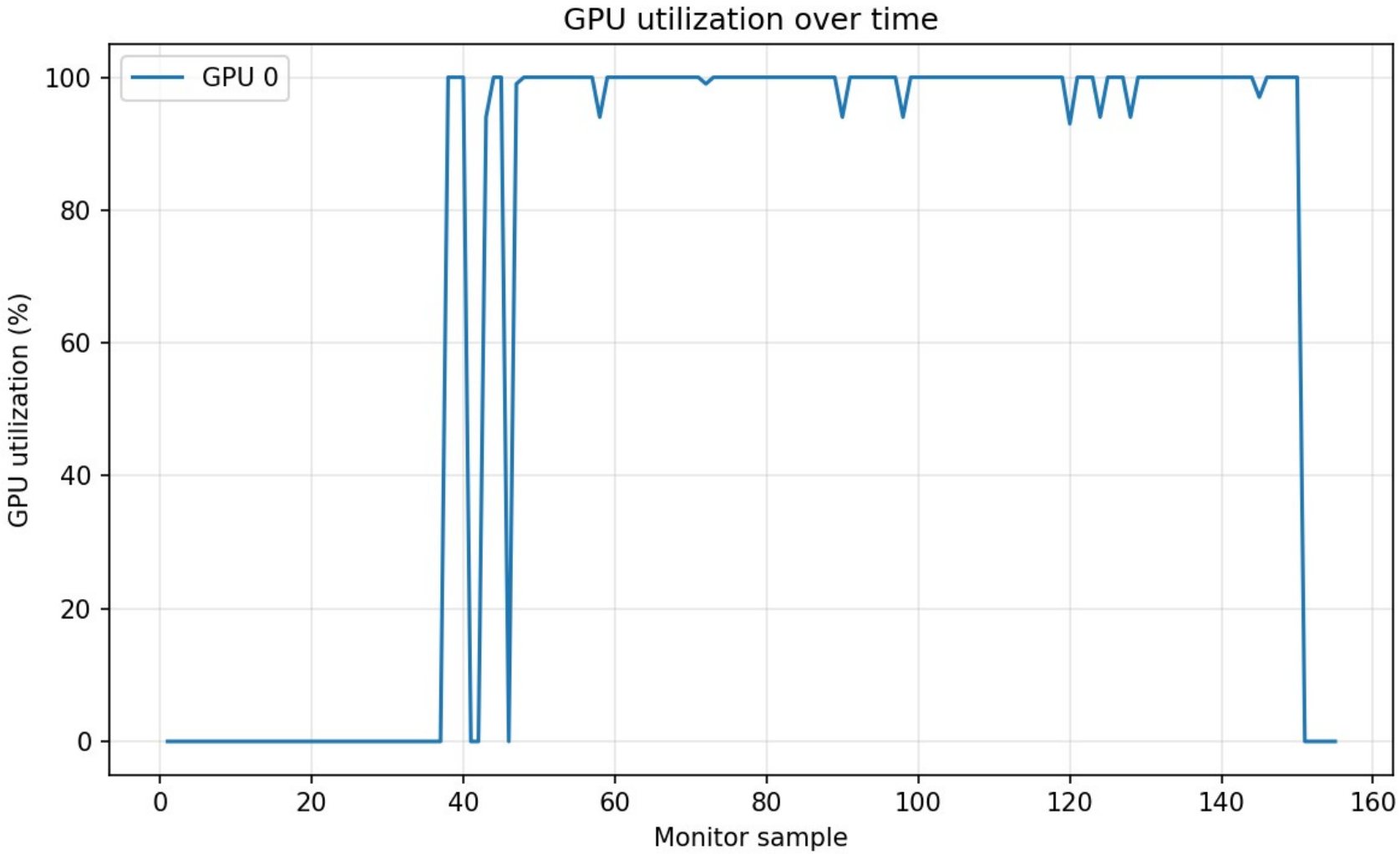
Baseline 2: DDP across two nodes

- PyTorch DistributedDataParallel via torchrun --nnodes=2.
  - DeepSpeed ZeRO-0 — no sharding, each rank holds the whole model.
- Per-device batch 16, effective batch 32, 313 steps for one epoch.
- Wall time: 74.8 min — about 1.3× faster than single GPU.
- Throughput: 2.22 samples/sec.
- Peak GPU memory: 30.7 GiB.
- GPU utilization: 70.7%.
  - Visible AllReduce stalls when both nodes wait on cross-node Ethernet.
- Final loss: 1.04. This is the reference everything else is compared against.



DDP — GPU utilization over time

Sharp drops on every step are AllReduce stalls — the cost of cross-node Ethernet.



Pipeline Parallelism

How we built it (Ghallab)

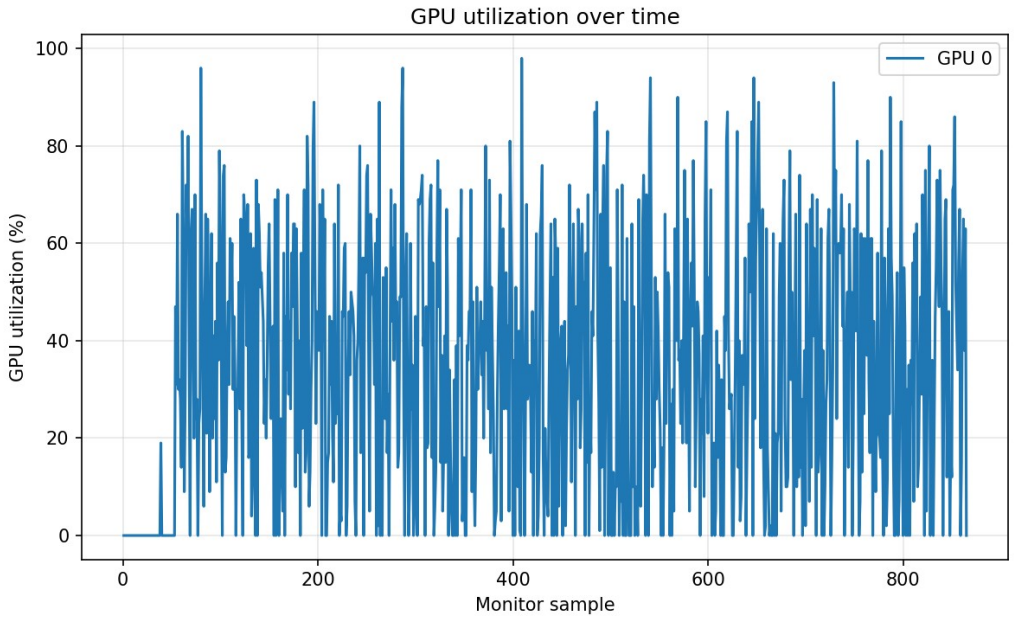
- Manual two-stage pipeline. We didn't use torchgpipe or DeepSpeed-Pipeline.
  - Custom training loop: LLaVA/llava/train/train_pipe_dist.py (~520 lines).
  - Gloo backend for cross-node send/recv. NCCL had ordering issues with our mixed-dtype activations.
- The model is cut at decoder layer k. There are 32 decoder layers in Vicuna-7B.
  - Rank 0 holds the vision tower (frozen), the MLP projection, and decoder layers 0..k-1.
  - Rank 1 holds layers k..31, the final norm, the LM head, and computes loss.
  - Activations flow forward, gradients flow back, by hand.
- Gradient accumulation 16, microbatch 1 → effective batch 16. LoRA is added on each rank's local layers.

Finding a split that fits

- Picking the layer index k matters more than expected.
- k = 16 (50/50 by layer count): rank 0 OOMs in the first forward.
  - Vision tower + image features sit on rank 0. With layers 0..15 it hits 31.4 GiB — over the 32 GiB card.
- k = 8 (push work to rank 1): rank 1 OOMs.
  - LM head + final norm + layers 8..31 hit 31.0 GiB.
- k = 12 with microbatch 1 + grad accum 16: both ranks fit.
  - Rank 1 ≈ 31.4 GiB, rank 0 ≈ 28 GiB. Run completes 625/625 steps.
- Lesson: layer count is a poor proxy for memory cost. The vision tower and LM head are heavier than they look.

Pipeline — results

- Wall time: 72.1 min. Almost identical to DDP.
- Throughput: 2.31 samples/sec — 1.04× DDP. Barely faster.
- Effective batch 16, half of DDP, so fewer steps per epoch.
- Peak GPU memory: 31.4 GiB on the heavier rank.
- GPU utilization is uneven: rank 0 at 34%, rank 1 at 56%.
  - Whichever rank finishes its layers first sits idle until the other catches up.
  - Gloo is much slower than NCCL, so the comm gap is wider.
- Final loss: 1.57 — higher than DDP because of the smaller effective batch.
- Pipeline made the model fit. It did not make it faster.



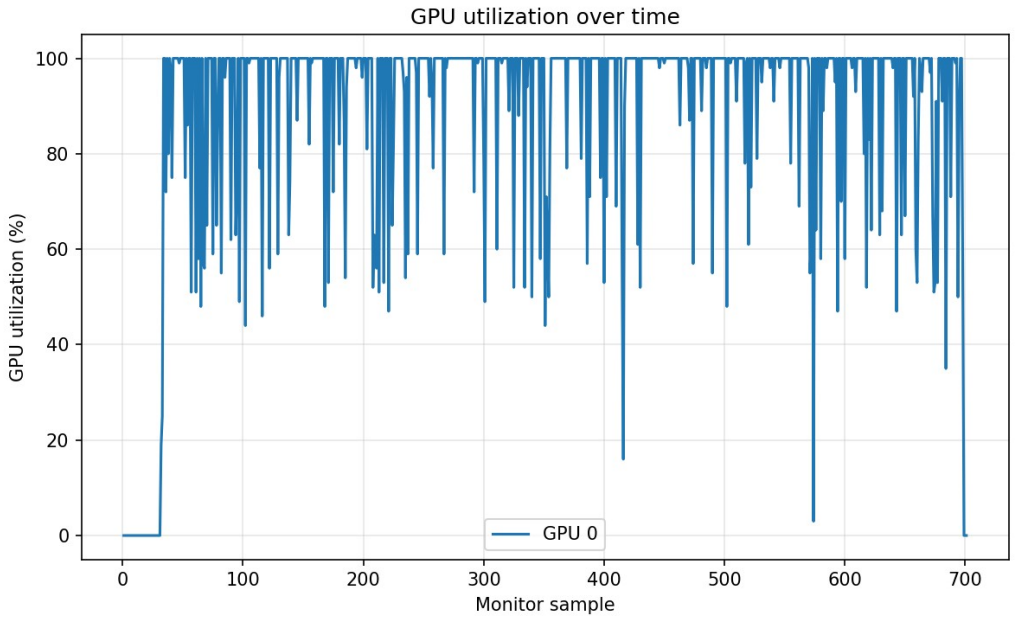
DeepSpeed ZeRO-2

What ZeRO-2 actually does (Omar)

- Still data parallelism: each rank trains on a different microbatch.
- What's sharded across the two ranks vs DDP:
  - Optimizer states (AdamW m, v) — sharded.
  - Gradients — sharded after backward via ReduceScatter.
  - Parameters — still replicated on every rank.
- Per-step communication:
  - Backward pass ReduceScatters the gradients to their owner-rank shards.
  - Each rank takes one optimizer step on its shard.
  - AllGather puts the updated parameters back on every rank.
- On paper this saves a lot of memory. On a LoRA workload it doesn't, because the trainable optimizer state is already small.
  - The real win comes from DeepSpeed's bucketed async overlap, not from sharding.

ZeRO-2 — results

- Wall time: 58.4 min — 16 minutes faster than DDP, 22% less wall-clock.
- Throughput: 2.85 samples/sec — 1.28× DDP.
- Peak GPU memory: 27.6 GiB. About 3 GiB lower than DDP.
- GPU utilization: 88.5% (DDP was 70.7%).
  - DeepSpeed's overlap fills the AllReduce-stall gap that DDP couldn't hide.
- Final loss: 1.11 — same as DDP within noise.
- Why isn't the speedup closer to 2×?
  - LoRA's optimizer state is tiny, so there isn't much to shard.
  - Most of the win is the better overlap, not the sharding itself.
- Even so: this is the only method in the comparison that beats plain DDP.



DeepSpeed ZeRO-3

ZeRO-3: shard everything (Rassul)

- Same data-parallel semantics as ZeRO-2. Each rank still sees a different microbatch.
- The difference: parameters are also sharded across the two ranks.
- Per-step comm now happens at every layer, twice:
  - Forward pass: AllGather parameters for layer N → compute → drop them.
  - Backward pass: AllGather them again → compute gradients → drop again.
  - ReduceScatter the gradients at the end of backward, then run the optimizer on each shard.
- On 2 ranks, ~7 GB of parameters live on each. But every layer briefly re-materializes its full ~430 MB on every rank.
- Back-of-envelope for our model:
  - 32 transformer layers × 430 MB × 2 (fwd + bwd) ≈ 27.5 GB transferred per step.
  - Over 1 Gbps Ethernet: roughly 4 minutes of comm per step before any compute.
- We used configs/zero3_bounded_async.json: bucketed allgather/reduce at 50 MB.

ZeRO-3 — results

- Hit the SLURM time limit before finishing. Reached 292 of 1250 steps in 3 h 52 min.
  - Each step took about 48 seconds. Projected full run: 16.5 hours.
- Throughput: 0.17 samples/sec — about 13× slower than DDP.
- Peak GPU memory: 22 GiB — actual sharding showing up.
  - But memory was never the bottleneck on this card. Communication was.
- GPU utilization: 99%. This is misleading.
  - DeepSpeed pipelines comm with compute, so 'utilization' includes time spent waiting on AllGather.
  - High util doesn't mean fast — it means the engine kept the GPU busy on something.
- Last logged loss: 1.31. Convergence was fine — the run was just too slow to finish.
- Diagnosis: per-layer AllGather over 1 Gbps Ethernet is the wall.

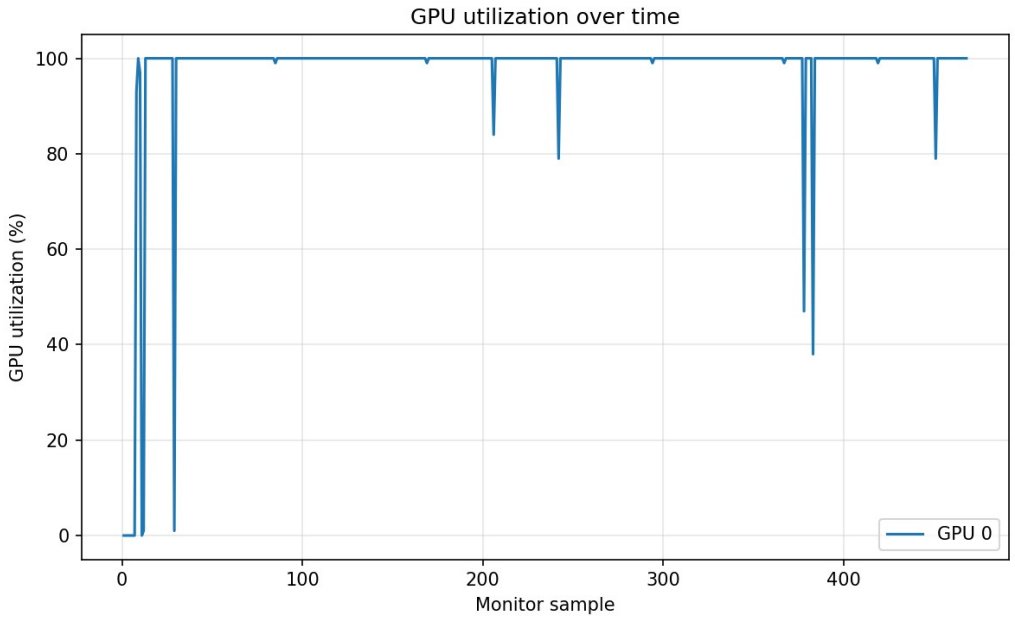
PyTorch FSDP

FSDP: PyTorch's take (Rassul)

- PyTorch-native equivalent of ZeRO-3. Same algorithm, different implementation.
- Design choices that differ from DeepSpeed:
  - Each transformer block is wrapped as one FSDP unit (a contiguous FlatParameter).
  - AllGather happens at the unit boundary, not per-tensor.
  - Mixed precision is set explicitly via a policy (we used bf16).
- Our config: full_shard auto_wrap on LlamaDecoderLayer, use_orig_params=True (required for LoRA), cpu_ram_efficient_loading=true.
- Two LLaVA-specific bugs we had to patch to get this running:
  - self.vision_tower[0] crashes when loading from a vendored checkpoint where vision_tower isn't a list.
  - LoRA adapters are created in fp32 after the base model is cast to bf16. FSDP requires uniform dtype, so we cast LoRA to bf16 too.

FSDP — results

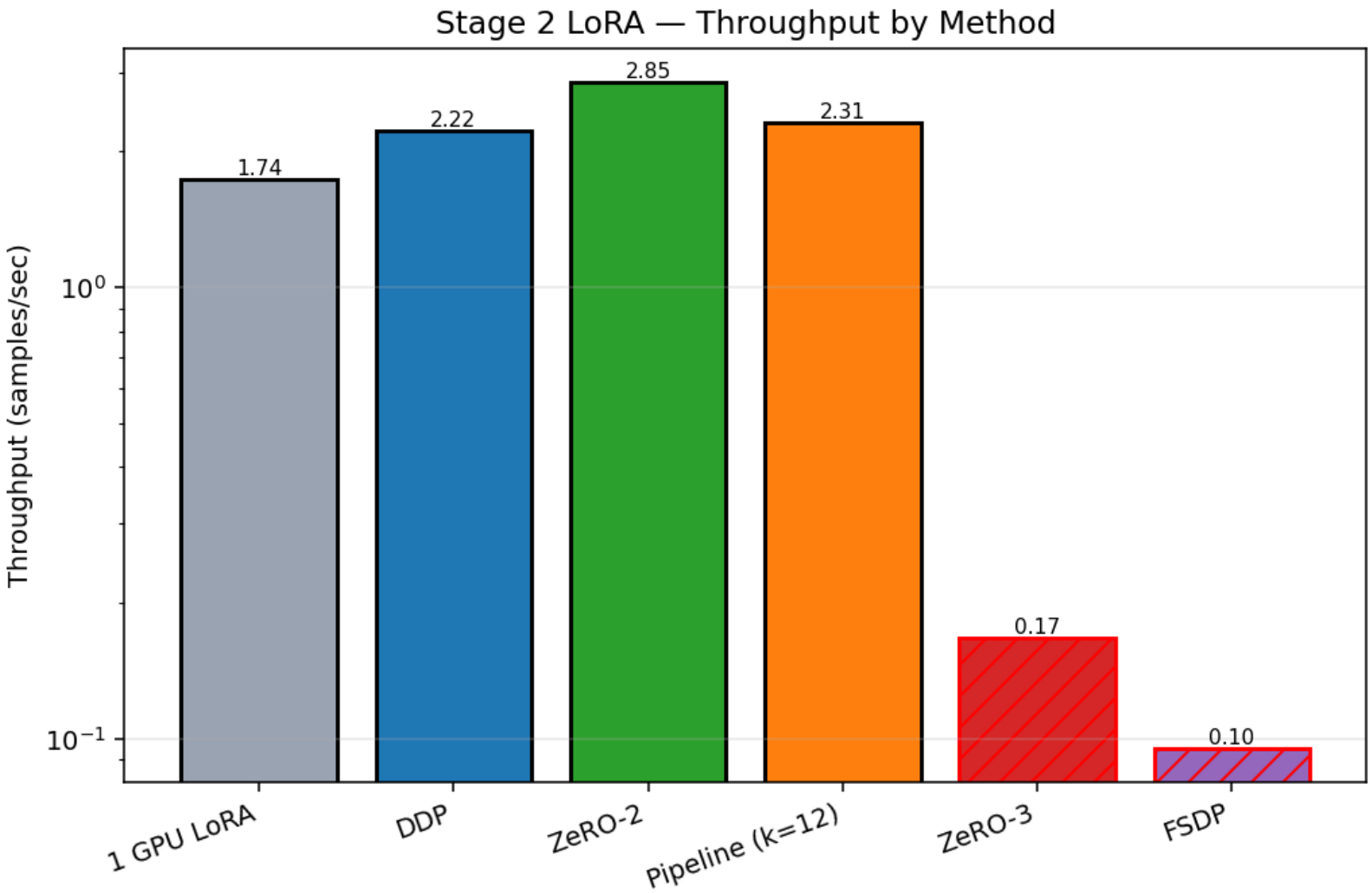
- Stopped early at step 28 of 1250 to capture metrics — same shape as the ZeRO-3 timeout.
- Wall time observed: 39 min. Each step took about 80 seconds.
  - Projected full run: 27.9 hours. Slower than ZeRO-3.
- Throughput: 0.095 samples/sec — about 22× slower than DDP, ~1.7× slower than ZeRO-3.
- Peak GPU memory: 32.3 GiB. Right at the card's limit.
  - FSDP's contiguous FlatParameter is bigger than DeepSpeed's bucket-based scheme.
- GPU utilization: 97.5%.
- Why slower than ZeRO-3 on the same hardware?
  - DeepSpeed's bounded async config (50 MB buckets) overlaps comm with compute aggressively.
  - FSDP's defaults wait for each transformer block's AllGather to finish before computing.
  - Same algorithm, different async behavior — the difference is real on a slow link.



Putting it all together

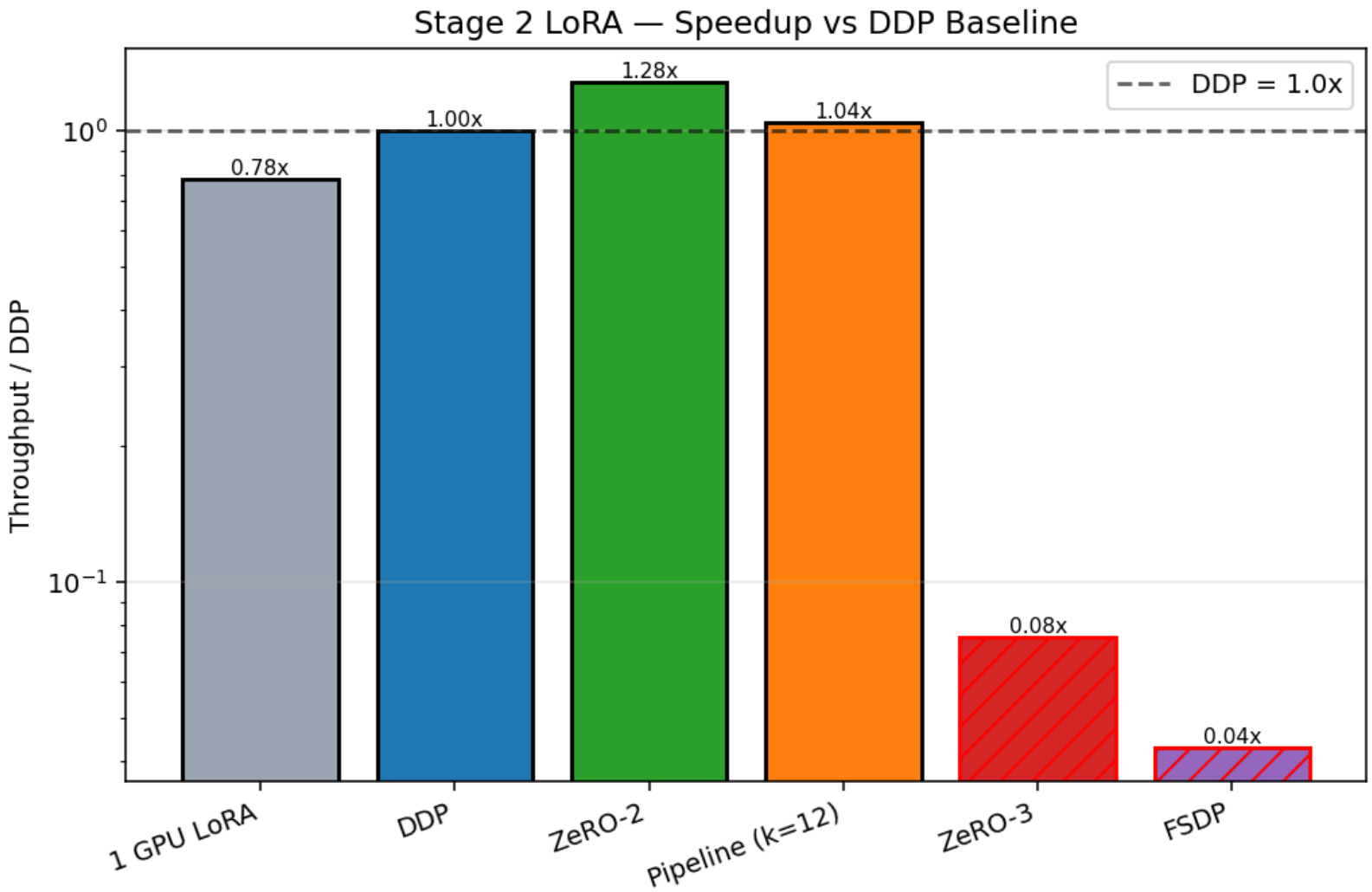
Throughput by method

ZeRO-2 wins. ZeRO-3 and FSDP collapse on the cross-node link.



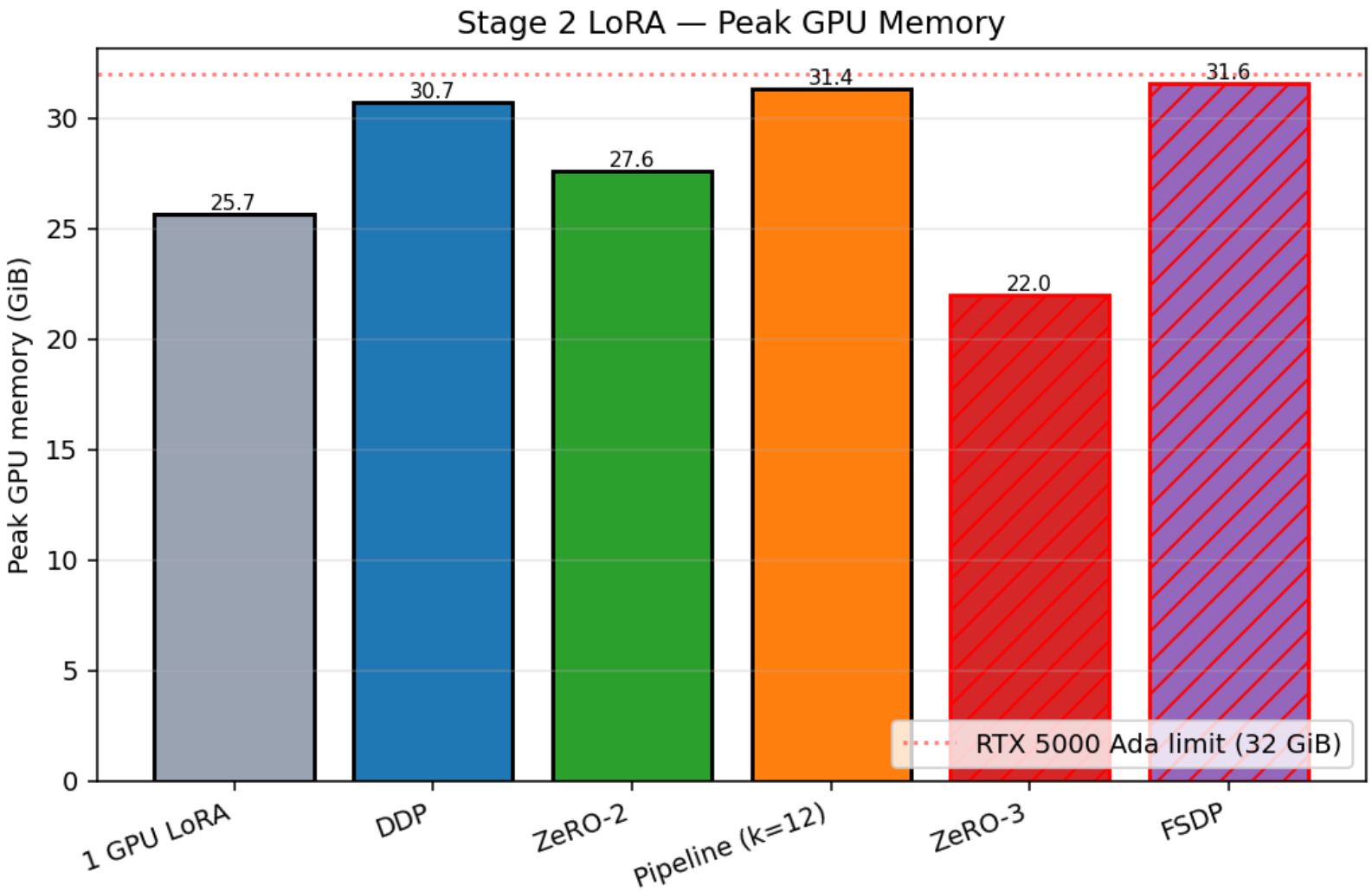
Speedup vs DDP

Only ZeRO-2 actually beats the naïve baseline.



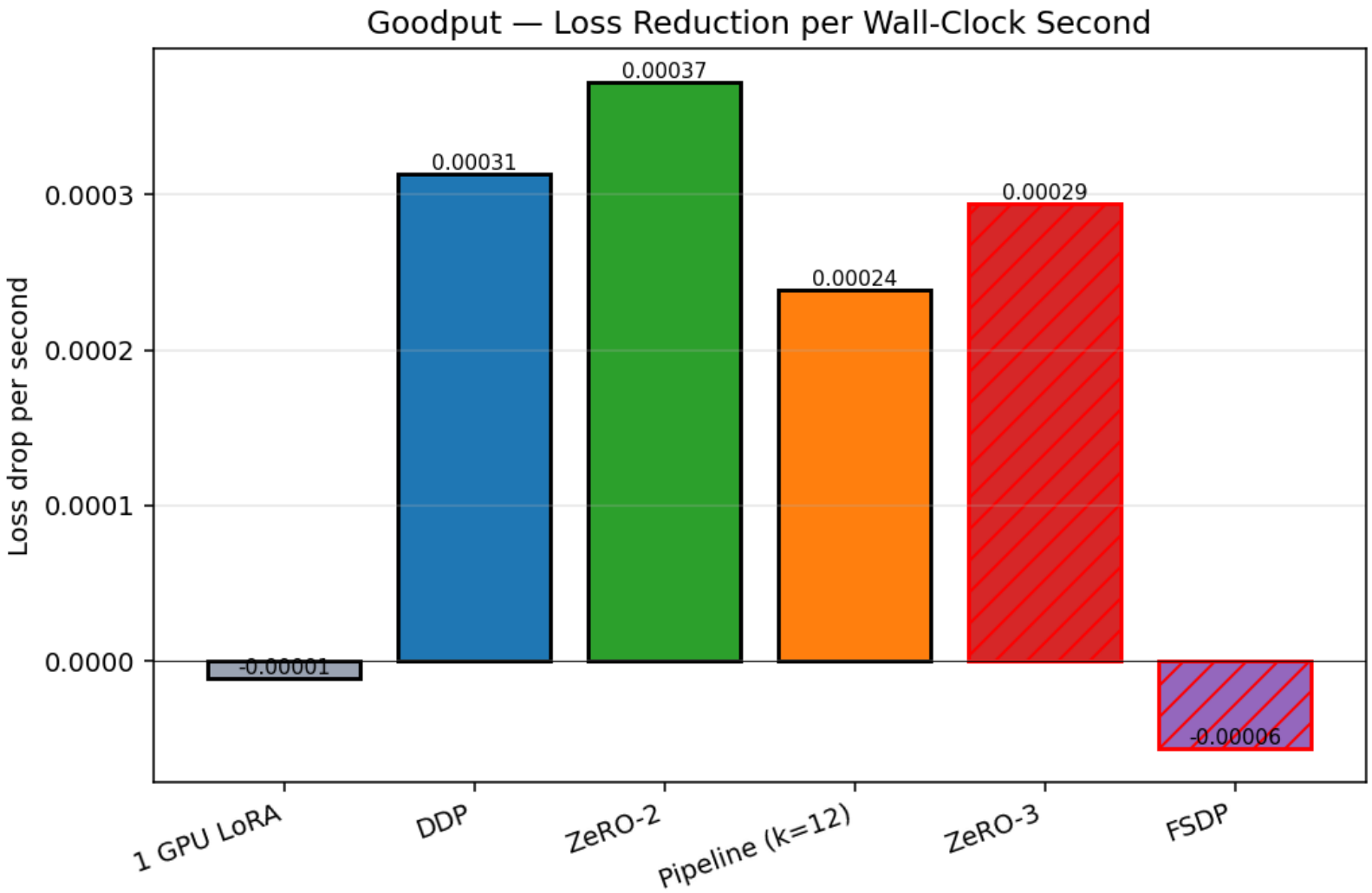
Peak GPU memory

ZeRO-3 shards the most, FSDP saturates the card. ZeRO-2 hits a sweet spot.



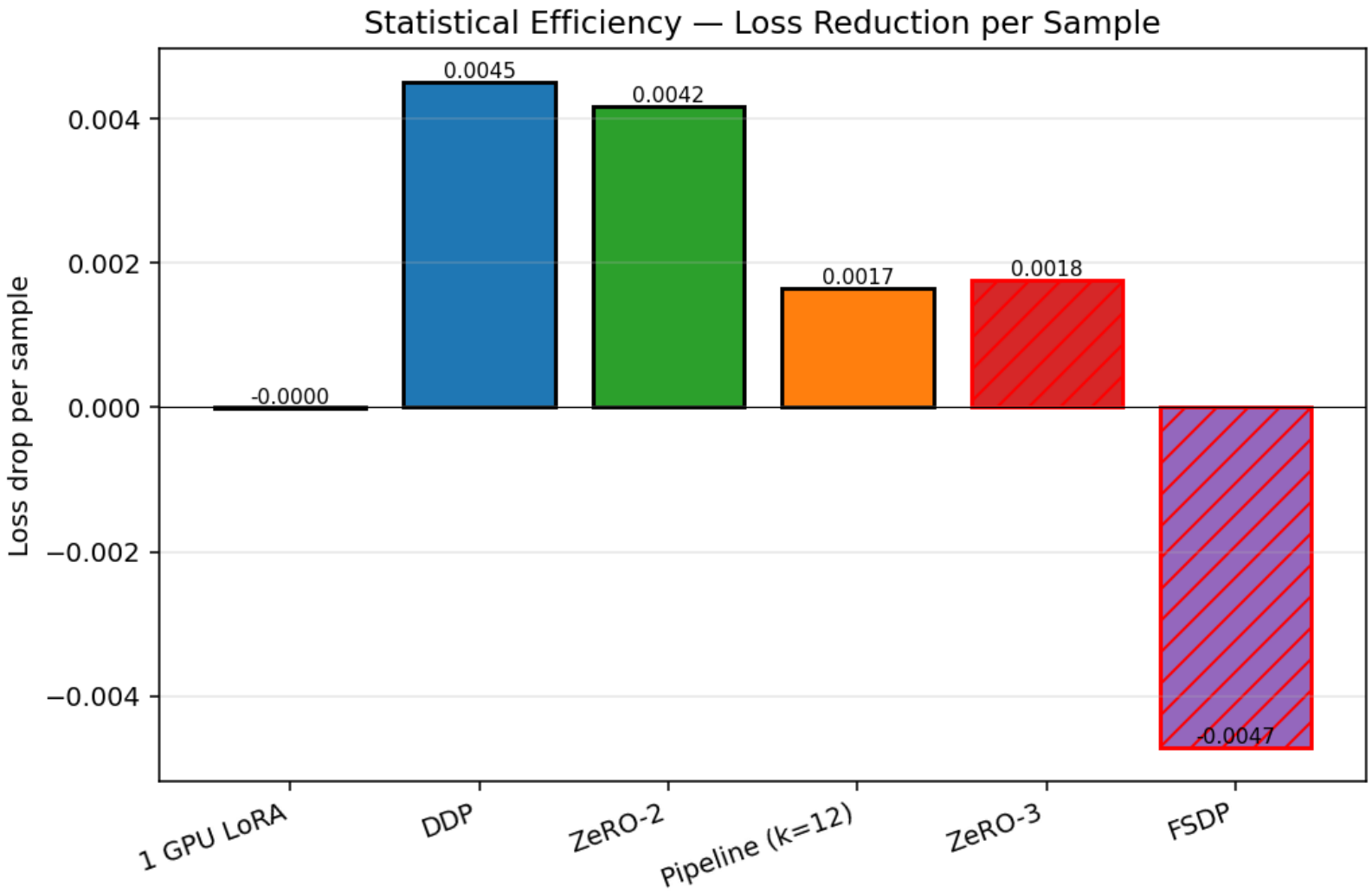
Goodput — loss reduction per second

What you get back from each second of GPU time.



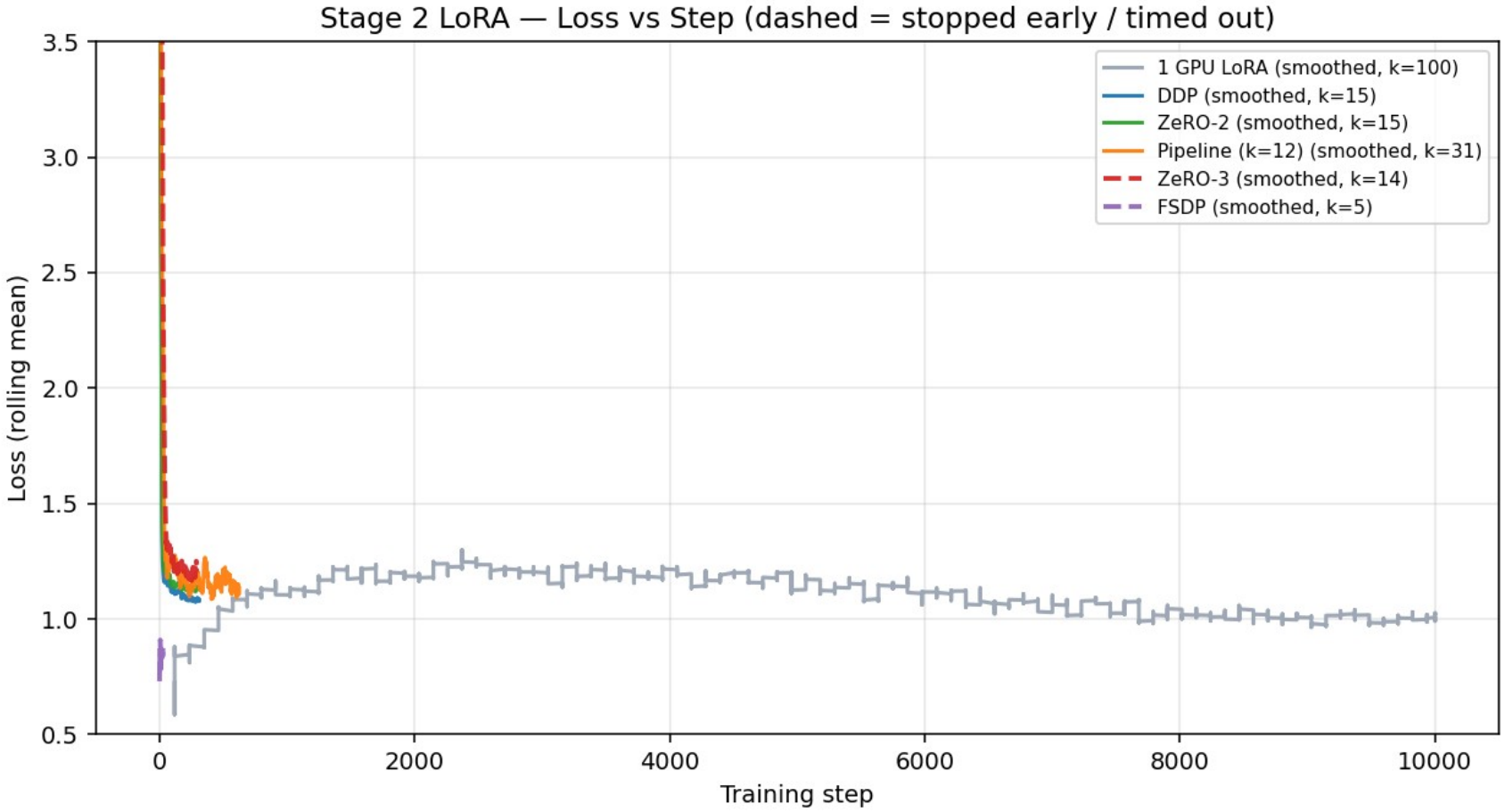
Statistical efficiency

Loss reduction per training sample. Larger effective batches converge faster per sample.



Loss vs step (smoothed)

Methods that completed all reach a similar loss. The gap is wall-clock, not statistical.



What we learned

- ZeRO-2 is the right tool for LoRA fine-tuning on slow interconnects.
  - Sharding optimizer + grads is enough. DeepSpeed's overlap quietly does the rest.
- Pipeline parallelism made the model fit but not faster.
  - Cross-node Gloo activation transfers are slower than DDP's single AllReduce.
  - Layer count is a poor proxy for memory cost — the vision tower and LM head dominate.
- ZeRO-3 and FSDP fall apart over Ethernet.
  - Per-layer AllGather of full bf16 weights is intractable on 1 Gbps.
  - Implementation matters — DeepSpeed ZeRO-3 is ~1.7× faster than PyTorch FSDP because of better overlap.
- These methods would scale on InfiniBand or NVLink. Wrong tool for this hardware.
- A 7B model with LoRA fits comfortably on a single 32 GiB card. Pick a data-parallel variant.

Thank you

Questions?

github.com/rassulmagauin/ml710 · ZERO2_RESULTS.md · results_explanation.md